

IIS SETUP & DEPLOYMENT — FULL RECORD

IIS SETUP

1. From where to download ASP.NET Core Hosting Bundle

You should download the ASP.NET Core Hosting Bundle from Microsoft's official .NET download page.

Microsoft .NET 8 Download — Official

Which one should you download?

First check your Employee Management project's .csproj file:

```
<TargetFramework>net8.0</TargetFramework>
```

If it says net8.0, go to the .NET 8 download page and look under:

ASP.NET Core Runtime → Windows → Hosting Bundle

Microsoft specifically recommends the Hosting Bundle for Windows/IIS because it includes the runtime and IIS support.

Important

Don't download only: .NET Runtime

or:

ASP.NET Core Runtime

For IIS deployment, choose:

ASP.NET Core Runtime



Windows



Hosting Bundle

The Hosting Bundle installs the ASP.NET Core Module for IIS, which allows IIS to host/forward requests to your ASP.NET Core application.

After installation

Restart IIS: iisreset

Then you can proceed with:

Publish Backend



Copy published files



Create IIS Application Pool



Create IIS Website



Test API

If you tell me what your project's <TargetFramework> says (net8.0, net9.0, net10.0, etc.), I can tell you exactly which Hosting Bundle to select.

2. I have downloaded IIS 8.0.30 then how to restart IIS?

Yes. If you've installed the **ASP.NET Core Hosting Bundle / IIS-related components**, you can restart IIS.

Easiest method — Command Prompt

1. Open **Command Prompt as Administrator**

2. Run: iisreset

You should see something like:

Attempting stop...

Internet services successfully stopped.

Attempting start...

Internet services successfully started.

That means IIS restarted successfully.

Or restart from IIS Manager

1. Press **Win + R**

2. Type: inetmgr

3. In the left panel, select your **computer/server name**

4. On the right side under **Actions**, click **Restart**

Important clarification

If by "IIS 8.0.30" you mean **IIS version 8.0.30**, that's not the same thing as the **ASP.NET Core Hosting Bundle**.

For your ASP.NET Core application, the important component is the **ASP.NET Core Hosting Bundle for the .NET version your application targets**.

If you tell me your project's target framework, for example:

<TargetFramework>net8.0</TargetFramework>

3.

In my appsettings.json i have this connection string:-

```
"ConnectionStrings": {  
  "DbConn":  
    "Server=localhost\\SQLEXPRESS;Database=Dapper;Trusted_Connection=True;TrustServerCertificate=True;"  
}
```

what changes need to do?

If your SQL Server and IIS are on the same Windows machine, then you may not need to change the connection string at all.

Your current connection string is:

```
"ConnectionStrings": {  
  "DbConn":  
    "Server=localhost\\SQLEXPRESS;Database=Dapper;Trusted_Connection=True;TrustServerCertificate=True;"  
}
```

The important part is:

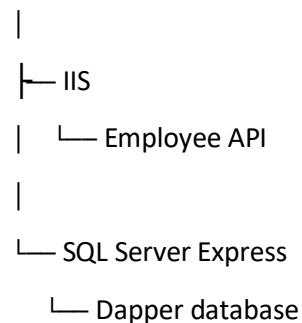
Server=localhost\SQLEXPRESS

When your application runs under IIS, localhost means **the IIS server itself**.

Scenario 1 — SQL Server is on the same machine as IIS

For example:

Your PC



Then you can **keep your connection string as it is**:

```
"ConnectionStrings": {
```

```
"DbConn":  
"Server=localhost\\SQLEXPRESS;Database=Dapper;Trusted_Connection=True;TrustServerCertificate=True;"  
}
```

However, there is one important issue: **Trusted_Connection=True` uses Windows Authentication.**

Your application is no longer running directly as your Windows user. It will run under an IIS Application Pool identity.

For example:

Your Windows account



Visual Studio



SQL Server



Works

But under IIS:

IIS



EmployeeManagementPool



ASP.NET Core



SQL Server

SQL Server needs to allow that IIS identity to access the **Dapper** database.

Recommended approach for your local IIS setup

If SQL Server Express and IIS are on the same machine, you can configure SQL Server permissions for your IIS Application Pool identity.

Suppose your Application Pool is:

EmployeeManagementPool

The Windows account you can grant access to is:

IIS APPPOOL\EmployeeManagementPool

In SQL Server Management Studio:

Security



Logins



New Login

Enter:

IIS APPPOOL\EmployeeManagementPool

Then map it to:

Dapper

and give it the database permissions your application requires.

For example, if this is your development/test server and the application needs CRUD access, you would grant appropriate database roles/permissions rather than changing the application to use an administrator account.

Scenario 2 — SQL Server is on another computer

Suppose your setup is:

Computer A



IIS



Employee API

Computer B



SQL Server Express



Dapper database

Then this:

localhost\SQLEXPRESS

is **wrong**.

You need to use the SQL Server machine's name/IP and instance.

For example:

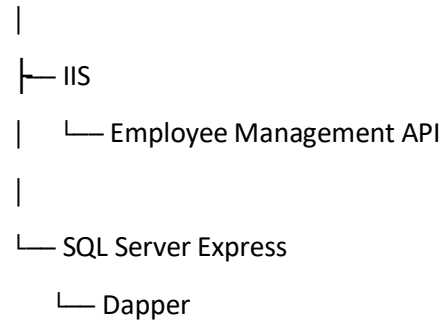
```
"ConnectionStrings": {  
  "DbConn":  
    "Server=192.168.1.100\\SQLEXPRESS;Database=Dapper;Trusted_Connection=True;TrustServerCertificate=True;"  
}
```

But Windows authentication across machines introduces additional configuration, so for a real server environment you would normally design the SQL authentication/service-account setup properly rather than simply putting credentials into `appsettings.json`.

Scenario 3 — IIS and SQL Server are on your own laptop

If you're doing this **just to learn IIS deployment on your ASUS laptop**, then your architecture is probably:

Your Laptop



In that case, **don't change** `localhost\\SQLEXPRESS` yet.

Keep:

```
"ConnectionStrings": {  
  "DbConn":  
    "Server=localhost\\SQLEXPRESS;Database=Dapper;Trusted_Connection=True;TrustServerCertificate=True;"  
}
```

The thing you need to configure is **SQL Server access for the IIS Application Pool identity**.

One more important thing: test it before IIS

Before deployment, verify that this database actually exists:

SQL Server

↓

SQLEXPRESS

↓

Dapper

In SSMS, run:

USE Dapper;

GO

SELECT DB_NAME();

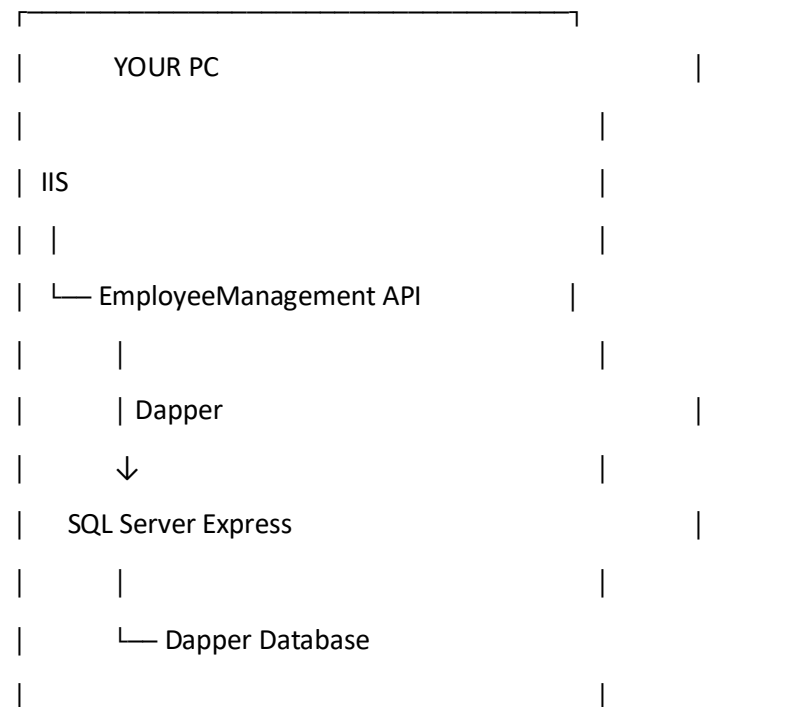
You should get:

Dapper

Then make sure your tables/stored procedures are there.

Your deployment architecture

For your current Employee Management System, if everything is on the same PC, I'd use:



So **the connection string itself can remain unchanged.**

The next thing I would configure is the **IIS Application Pool identity's SQL Server permissions**, because that is the part most likely to cause:

Login failed for user 'IIS APPPOOL\EmployeeManagementPool'

when you first run the API through IIS.

4. Where i can get security -> login -> new login in ssms tell in detail

Yes. This is done **inside SQL Server Management Studio (SSMS)**, not inside IIS.

For your current setup, we'll assume:

```
IIS
↓
EmployeeManagementPool
↓
ASP.NET Core API
↓
SQL Server Express
↓
Dapper database
```

Your connection string uses:

```
Trusted_Connection=True
```

so we need to give the IIS Application Pool identity permission to access the **Dapper** database.

Step 1 — Open SSMS

Open **SQL Server Management Studio**.

You will get:

Connect to Server

Use the same SQL Server instance that your application currently uses.

For your connection string:

```
Server=localhost\SQLEXPRESS
```

In SSMS, enter:

```
Server type:      Database Engine
Server name:      localhost\SQLEXPRESS
Authentication:   Windows Authentication
```

Then click **Connect**.

Step 2 — Find your SQL Server instance

After connecting, look at the left side:

```
Object Explorer
└─ localhost\SQLEXPRESS
```

Expand it:

```
localhost\SQLEXPRESS
├─ Databases
├─ Security
├─ Server Objects
├─ Replication
└─ Management
```

You will see **Security** directly underneath your server.

📌 **Important:** There are two different **Security** sections you may encounter.

We want the **server-level Security**:

```
localhost\SQLEXPRESS
    ↓
Security
```

NOT:

```
Databases
    ↓
Dapper
    ↓
Security
```

For creating the login, use the first one.

Step 3 — Expand Security

Click the arrow next to:

```
Security
```

You should see:

```
Security
├─ Logins
├─ Server Roles
└─ Credentials
```

└─ Audits

Click the arrow next to:

Logins

You will see existing SQL Server logins.

For example:

```
Logins
├─ sa
├─ BUILTIN\Administrators
├─ NT AUTHORITY\SYSTEM
└─ ...
```

Step 4 — Right-click Logins

Right-click:

Logins

You should get a context menu containing:

New Login...

Click:

New Login...

A window called:

Login - New

will open.

Step 5 — Enter the IIS Application Pool identity

This is the important part.

At the top you will see:

Login name:

Enter:

IIS APPPOOL\EmployeeManagementPool

assuming your IIS Application Pool is called:

```
EmployeeManagementPool
```

If you named your Application Pool something else, use that exact name.

For example, if your pool is:

```
EmployeeAPI-Pool
```

then enter:

```
IIS APPPOOL\EmployeeAPI-Pool
```

Step 6 — Don't use SQL Server Authentication

You will see two options:

- ☐ Windows authentication
- ☐ SQL Server authentication

Select:

```
Windows authentication
```

Because your connection string says:

```
Trusted_Connection=True
```

You **do not need to create a username/password** here.

Step 7 — Go to User Mapping

On the left side of the **Login - New** window, you'll see:

```
General
Server Roles
User Mapping
Securables
Status
```

Click:

User Mapping

You'll see a list of databases.

Something like:

```
Map
Database
```

- ☐ master
- ☐ model
- ☐ msdb
- ☐ tempdb
- ☐ Dapper

Find:

Dapper

and tick the checkbox:

☒ Dapper

Step 8 — Give the login database permissions

After selecting Dapper, look at the bottom section:

Database role membership for: Dapper

You'll see roles such as:

- ☐ db_datareader
- ☐ db_datawriter
- ☐ db_ddladmin
- ☐ db_owner
- ☐ public

For your **local learning/development Employee Management System**, you can initially select:

- ☒ db_datareader
- ☒ db_datawriter
- ☒ public

This gives your application:

SELECT
INSERT
UPDATE
DELETE

access.

Don't select db_owner unnecessarily

You may see tutorials telling you to select:

☒ db_owner

It will usually make the application work, but it gives extremely broad permissions.

For a proper application, follow **least privilege**.

For your current IIS learning setup, `db_datareader` + `db_datawriter` is a reasonable starting point.

Step 9 — Click OK

Click:

OK

SSMS should create the login.

Now go back to:

```
localhost\SQLEXPRESS
  ↓
Security
  ↓
Logins
```

You should see:

```
IIS APPPOOL\EmployeeManagementPool
```

Step 10 — Verify the database user

Now expand:

```
Databases
  ↓
Dapper
  ↓
Security
  ↓
Users
```

You should find a user corresponding to your IIS application pool login.

Conceptually:

```
Dapper
├── Security
│   └── Users
│       └── IIS APPPOOL\EmployeeManagementPool
```

That's the relationship we want:

```
IIS Application Pool
```

```
↓
IIS APPPOOL\EmployeeManagementPool
↓
SQL Server Login
↓
Dapper Database User
↓
db_datareader / db_datawriter
```

Step 11 — Check your IIS Application Pool name

Before doing this, make sure the name is exactly correct.

Open:

```
IIS Manager
↓
Application Pools
```

You might see:

```
Name
-----
.NET v4.5
DefaultAppPool
EmployeeManagementPool
```

If your application pool is:

```
EmployeeManagementPool
```

then your SQL Server login must be:

```
IIS APPPOOL\EmployeeManagementPool
```

Spelling and spaces matter.

Step 12 — Your connection string remains the same

After doing this, you can continue using:

```
{
  "ConnectionStrings": {
    "DbConn":
    "Server=localhost\\SQLEXPRESS;Database=Dapper;Trusted_Connection=True;TrustServerCertificate=True;"
  }
}
```

You don't need to put:

```
Username
Password
```

into the connection string.

The authentication flow becomes:

```
Employee API
↓
IIS Application Pool
↓
EmployeeManagementPool
↓
Windows Identity
↓
SQL Server
↓
Dapper Database
```

One important check before you do this

If you **cannot find**:

```
IIS APPPOOL\EmployeeManagementPool
```

in the **New Login** dialog, don't panic. This is a common point of confusion.

You can click **Search...** next to Login name and then enter the application pool identity, or we can create it using a SQL command.

In fact, for IIS application pool identities, I generally prefer using the exact identity:

```
CREATE LOGIN [IIS APPPOOL\EmployeeManagementPool] FROM WINDOWS;
```

and then map it to your **Dapper** database.

If you want, I can walk you through **the exact SSMS GUI clicks from `New Login` → `User Mapping` → `db\_datareader/db\_datawriter`**, including what every field on each screen means.

Publish your backend

In Visual Studio:

Solution Explorer

↓

Right-click your ASP.NET Core backend project

↓

Publish...

You should get the Publish screen.

Select:

Target



Folder

Then choose a folder such as:

C:\Publish\EmployeeManagement

Set:

Configuration: Release

Then click:

Publish

You should eventually get something like:

C:\Publish\EmployeeManagement

|

|— EmployeeManagement.dll

|— EmployeeManagement.deps.json

|— EmployeeManagement.runtimeconfig.json

|— web.config

|— appsettings.json

|— appsettings.Development.json

|— wwwroot

The exact files depend on your project.

Yes. **A custom folder is perfectly fine, and for IIS deployment I actually recommend it.**

Your current default:

| bin\Release\net8.0\publish\

is valid, but it's inside your project directory.

I recommend using:

| C:\Publish\EmployeeManagement

So in the **Folder location** field shown in your screenshot, replace:


```
bin\Release\net8.0\publish\
```

with:

```
C:\Publish\EmployeeManagement
```

Then click **Finish**.

Why use C:\Publish\EmployeeManagement?

It gives you a clean separation:

```
Your Visual Studio Project
C:\...\EmployeeManagement\
    ↓
    ↓ Publish
    ↓
C:\Publish\EmployeeManagement\
    ↓
    ↓ IIS points here
    ↓
IIS → EmployeeManagement Website
```

This is cleaner than having IIS point directly into:

```
YourProject\bin\Release\net8.0\publish
```

It also makes future deployments easier:

```
C:\Publish\EmployeeManagement
```

can be your dedicated deployment directory.

One important point

You **do not need to create** `C:\Publish\EmployeeManagement` **manually first**. Visual Studio can create the folder during publishing if it doesn't exist.

So, on your current screen:

Folder location:

```
C:\Publish\EmployeeManagement
```

Then:

Finish → Publish

After that, we will check that the folder contains files such as:

```
EmployeeManagement.dll
web.config
appsettings.json
EmployeeManagement.deps.json
EmployeeManagement.runtimeconfig.json
```

Don't create the IIS website yet. First complete the publish successfully. Then we'll create the Application Pool and IIS website in the correct order.

Published now next step

Good. Now that the backend is published, let's proceed in the correct order.

Step 1 — Verify the published folder

Open:

```
C:\Publish\EmployeeManagement
```

You should see files similar to:

```
EmployeeManagement.dll
EmployeeManagement.deps.json
EmployeeManagement.runtimeconfig.json
web.config
appsettings.json
```

You may also have:

```
wwwroot
```

If those are there, the publish is successful.

Step 2 — Create the IIS Application Pool

Open **IIS Manager**:

```
Win + R
```

type:

```
inetmgr
```

and press Enter.

On the left:

```
Your Computer
↓
Application Pools
```

Right-click **Application Pools** → **Add Application Pool...**

Enter:

Name

```
EmployeeManagementPool
```

.NET CLR Version

Select:

No Managed Code

Managed pipeline mode

Keep:

Integrated

Start application pool immediately

Keep it checked.

Then click **OK**.

You should now see:

Application Pools
└─ EmployeeManagementPool

Step 3 — Create the IIS Website

Now go to:

IIS Manager
↓
Sites

Right-click **Sites** → **Add Website...**

Use:

Setting	Value
Site name	EmployeeManagement
Physical path	C:\Publish\EmployeeManagement
Application Pool	EmployeeManagementPool
Type	http
IP Address	All Unassigned
Port	8080
Host name	Leave blank

So it should effectively be:

Site name:
EmployeeManagement

```
Physical path:  
C:\Publish\EmployeeManagement
```

```
Application Pool:  
EmployeeManagementPool
```

```
Binding:  
http  
Port:  
8080
```

Click **OK**.

Step 4 — Start the website

Under:

```
Sites  
└─ EmployeeManagement
```

make sure the website is **Started**.

Also go to:

```
Application Pools  
└─ EmployeeManagementPool
```

and make sure it is **Started**.

Step 5 — Don't configure SQL Server yet

We now know the exact IIS Application Pool name:

```
IIS APPPOOL\EmployeeManagementPool
```

Now we can go back to SSMS and configure the SQL Server login.

But before doing that, there's one useful test.

Open your browser on the same computer and go to:

```
http://localhost:8080
```

Because this is an API, you may get something like:

```
404
```

or a blank response. **That doesn't automatically mean IIS is broken.**

If your application has Swagger enabled in the published environment, try:

```
http://localhost:8080/swagger
```

If Swagger isn't available in production, we'll test an actual endpoint, for example:

```
http://localhost:8080/api/Employee
```

If you get a 500.30 / 500.31 / 502.5

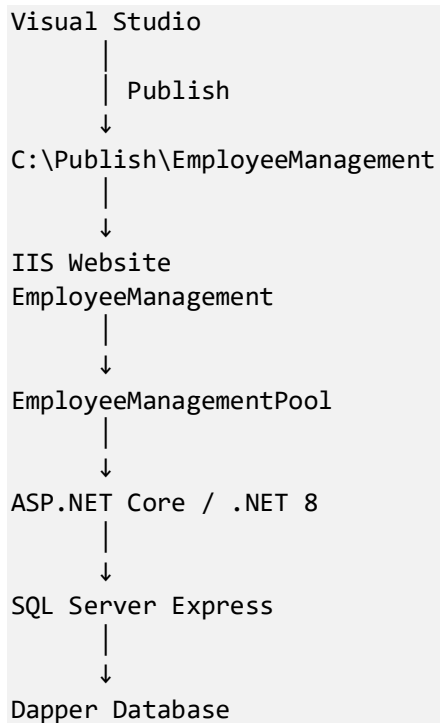
Don't start changing random IIS settings.

Those errors usually tell us something specific about:

- * .NET runtime/Hosting Bundle
- * Application startup
- * appsettings.json
- * Database connection
- * Permissions
- * Application configuration

We can diagnose the exact error.

Your current setup should now look like:



Do Steps 2–4 first. Once the IIS Website and EmployeeManagementPool are created, tell me what happens when you open `http://localhost:8080`.

Trying to identify which one is the application pool?

You're currently on the **server Home** screen in IIS Manager. The Application Pool is not visible in the main panel.

Click this on the right side

In your screenshot, under **Actions**, you have:

`View Application Pools`

Click that.

Alternatively, on the left side:

1. Click the small arrow next to your computer name:

LAPTOP-ED55U690

2. You should see:

Application Pools
Sites

3. Click **Application Pools**.

You will then see the list of application pools.

What to do next

If you haven't created one yet, you'll probably see:

.NET v4.5
DefaultAppPool

Then:

Right-click `Application Pools` → `Add Application Pool...`

Create:

Name:
EmployeeManagementPool

.NET CLR Version:
No Managed Code

Managed Pipeline Mode:
Integrated

Click **OK**.

Then you'll have:

```
Application Pools
├── .NET v4.5
├── DefaultAppPool
└── EmployeeManagementPool ← your pool
```

Don't create the SQL Server login yet. First create this pool. Then we'll use its exact name (EmployeeManagementPool) when configuring SQL Server.

After creating application pool I am seeing like this

Is it correct?

Yes. **This is correct.** Your Application Pool has been created successfully.

From your screenshot:

Setting	Your value	Correct?
Application Pool	Dapper_CRUD	✓
Status	Started	✓
.NET CLR Version	No Managed Code	✓
Managed Pipeline	Integrated	✓
Applications	0	✓for now

The **Applications = 0** is normal because you haven't associated your IIS website with this pool yet.

Your next step: Create the IIS Website

You now have:

```
Application Pool
└── Dapper_CRUD
```

Now on the **left side**, right-click:

```
Sites
```

and select:

```
Add Website...
```

You'll get the **Add Website** window.

Enter:

```
Site name:
Dapper_CRUD
```

For **Physical path**, use the folder where you published your backend, for example:

```
C:\Publish\EmployeeManagement
```

For **Application Pool**, select:

```
Dapper_CRUD
```

For the binding:

```
Type:      http
IP address: All Unassigned
Port:      8080
Host name: [leave blank]
```

So your configuration will look like:

```
Add Website

Site name:      Dapper_CRUD
Physical path:   C:\Publish\EmployeeManagement
Application pool: Dapper_CRUD

Binding:
Type:           http
IP address:     All Unassigned
Port:           8080
Host name:      [blank]
```

Then click **OK**.

After that

Your IIS structure should look approximately like:

```
LAPTOP-ED55U690
├── Application Pools
│   ├── Dapper_CRUD      ← You created this
│   └── DefaultAppPool
└── Sites
    ├── Default Web Site
    └── Dapper_CRUD      ← We'll create this now
```

Go ahead and click `Sites → Add Website...`. If you send me a screenshot of the **Add Website** window, I'll tell you exactly what to put in each field.

The same has been created no tell me next step

Good. If you now have both **Application Pool `Dapper\_CRUD`** and **IIS Site `Dapper\_CRUD`** created, the next step is **SQL Server permission for the IIS Application Pool**.

Your setup is now:

```
IIS
├── Application Pools
│   └── Dapper_CRUD
└── Sites
```



```
└─ Dapper_CRUD
   └─ C:\Publish\EmployeeManagement
```

Step 1 — Verify the IIS site's Application Pool

In IIS:

Sites → Dapper_CRUD

Then on the right click **Basic Settings...**

Check that:

```
Application pool: Dapper_CRUD
Physical path:    C:\Publish\EmployeeManagement
```

If both are correct, click **Cancel/OK**.

Step 2 — Give IIS access to the published folder

This is important.

Go to Windows Explorer:

```
C:\Publish
```

Right-click:

```
EmployeeManagement
```

→ **Properties**

→ **Security**

→ **Edit**

→ **Add**

Enter:

```
IIS AppPool\Dapper_CRUD
```

Click **Check Names**.

It should resolve to something like:

```
IIS APPPOOL\Dapper_CRUD
```

Give it:

```
Read & execute
List folder contents
```

Read

You generally don't need to give it **Full Control**.

Click **Apply** → **OK**.

Step 3 — Now configure SQL Server

This is where your earlier SSMS screen comes into play.

Open SSMS and connect to:

localhost\SQLEXPRESS

Go to:

Security
↓
Logins

Right-click **Logins** → **New Login...**

For **Login name**, enter:

IIS APPPOOL\Dapper_CRUD

Select:

Windows authentication

Then click **User Mapping** on the left.

Find:

Dapper

and tick it:

☒ Dapper

At the bottom, select:

☒ db_datareader
☒ db_datawriter
☒ public

Then click **OK**.

Your authentication flow will now be:

Browser / Angular
↓
IIS
↓

```
Dapper_CRUD Application Pool
↓
IIS APPPOOL\Dapper_CRUD
↓
SQL Server
↓
Dapper Database
```

And your existing connection string can remain:

```
"ConnectionStrings": {
  "DbConn":
  "Server=localhost\\SQLEXPRESS;Database=Dapper;Trusted_Connection=True;TrustServerCertificate=True;"
}
```

Step 4 — Restart the Application Pool

In IIS:

```
Application Pools
↓
Dapper_CRUD
```

Right-click → **Recycle**.

Or use **Stop**, then **Start**.

Step 5 — Test the API

Now open a browser and try:

```
http://localhost:8080
```

Since this is an API, don't worry if / gives a 404.

Try your actual endpoint, for example:

```
http://localhost:8080/api/Employee
```

Or, if Swagger is enabled in the published application:

```
http://localhost:8080/swagger
```

What we want to see

If:

```
/swagger
```

opens → IIS + ASP.NET Core hosting is working.

If you get:

```
500.30
500.31
502.5
```

or another error, **don't change anything randomly**. Send me the exact error and we'll fix that specific issue.

Do Steps 1–5 in order. The most important next action right now is creating the `IIS APPPOOL\Dapper\_CRUD` SQL Server login.

After clicking on Ok a message pops up :-

=====

You must enter a Login name. (SqlManagerUI)

Program Location:

at
Microsoft.SqlServer.Management.SqlManagerUI.CreateLoginGeneral.OnGatherUiInformation(RunType runType)

at
Microsoft.SqlServer.Management.SqlMgmt.ViewSwitcherControlsManager.OnGatherUiInformation(RunType runType)

That error means **the Login name field in the SSMS `New Login` window was left empty**. My previous instruction was too compressed.

You are at the **Login - New** window. Let's do it carefully.

1. Go back to the General tab

On the left side, click:

General

You should see a field near the top:

Login name: [_____]

Enter exactly:

```
IIS APPPOOL\Dapper_CRUD
```

So it should look like:

```
Login name: IIS APPPOOL\Dapper_CRUD
```

2. Authentication

Under that, select:

Windows authentication

Do **not** select SQL Server authentication.

3. Important: Don't use Search... yet

Just type the following directly into **Login name**:

IIS APPPOOL\Dapper_CRUD

You don't need to enter a password.

4. Then click User Mapping

On the left:

General
Server Roles
User Mapping ← click this
Securables
Status

Find:

Dapper

and check:

☒ Dapper

At the bottom, select:

☒ db_datareader
☒ db_datawriter
☒ public

5. Then click OK

Now SSMS should accept it.

If IIS APPPOOL\Dapper_CRUD gives another error

That's okay. **Don't change anything else.**

Send me a screenshot of your **Login - New → General** screen before clicking OK. I can see exactly what field is missing and tell you what to enter.

In SSMS in Object Explorer Inside Security => Logins now i can see IIS APPPOOL\Dapper_CRUD

is it correct now tell me the next step?

Yes. **That is correct.** Seeing:

```
Security
└─ Logins
    └─ IIS APPPOOL\Dapper_CRUD
```

means the **SQL Server login has been created successfully**. ✓

Now we need to make sure this login has access to your **Dapper** database.

Step 1 — Verify Dapper database mapping

In SSMS:

```
Security
  ↓
Logins
  ↓
IIS APPPOOL\Dapper_CRUD
```

Right-click `IIS APPPOOL\Dapper\_CRUD` → Properties

A **Login Properties** window will open.

On the left, click:

```
User Mapping
```

You should see your databases.

Find:

```
Dapper
```

Make sure this is checked:

```
☒ Dapper
```

Then look at the bottom:

```
Database role membership for: Dapper
```

Make sure you have:

```
☒ public
☒ db_datareader
☒ db_datawriter
```

For your current local IIS learning setup, this is sufficient for normal CRUD operations.

Then click **OK**.

Step 2 — Check the published folder permissions

Now go to:

```
C:\Publish\EmployeeManagement
```

Right-click the folder:

Properties → Security → Edit → Add

Enter:

```
IIS APPPOOL\Dapper_CRUD
```

Click **Check Names**.

It should resolve successfully.

Give it:

```
Read & execute  
List folder contents  
Read
```

Then:

Apply → OK → OK

This allows IIS to read your published application files.

Step 3 — Verify your IIS Website

Open IIS Manager:

```
Sites  
↓  
Dapper_CRUD
```

Right-click:

Manage Website → Advanced Settings...

Check:

```
Application Pool = Dapper_CRUD
```

Also check the physical path is:

```
C:\Publish\EmployeeManagement
```

If those are correct, you're good.

Step 4 — Check Application Pool

Go to:

```
Application Pools
↓
Dapper_CRUD
```

It should show:

```
Status:                Started
.NET CLR Version:      No Managed Code
Managed Pipeline:     Integrated
```

Your screenshot earlier already showed these settings correctly. ✓

Step 5 — Recycle the Application Pool

Right-click:

```
Dapper_CRUD
```

and select:

Recycle

Wait a few seconds.

Step 6 — Test your API

Now open your browser.

Since you configured port 8080, first try:

```
http://localhost:8080
```

Because this is an API, a 404 at the root URL isn't necessarily a problem.

Better: test your actual API endpoint

For example, if your controller is:

```
[Route("api/[controller]")]
public class EmployeeController : ControllerBase
```

try:

```
http://localhost:8080/api/Employee
```


Or, if Swagger is enabled:

```
http://localhost:8080/swagger
```

What we're looking for

If Swagger opens:

```
http://localhost:8080/swagger
```

then your chain is working:

```
Browser
  ↓
IIS
  ↓
Dapper_CRUD Application Pool
  ↓
ASP.NET Core .NET 8
  ↓
Dapper
  ↓
SQL Server Express
```

If you get an error such as:

```
500.30
500.31
502.5
500
```

don't make further changes yet. Send me the exact error/screenshot. We can identify whether it is the .NET Hosting Bundle, application startup, database permission, connection string, or IIS configuration.

One thing I want you to check first

Before testing the API, go into:

`IIS APPPOOL\Dapper\_CRUD` → Properties → User Mapping

and confirm that **`Dapper` is checked**. That is the next immediate step.

I have entered there IIS APPPOOL\Dapper_CRUD but it is coming Dapper_CRUD only

is that ok?

any error will be there?

Yes — **that is correct and there is no error.** You're seeing two different names because SQL Server distinguishes between the **server login** and the **database user**.

What you entered

At the server level, you created:

```
IIS APPPOOL\Dapper_CRUD
```

You can see this under:

```
Security
  → Logins
    → IIS APPPOOL\Dapper_CRUD
```

✔ This is correct.

Why User Mapping shows only Dapper_CRUD

When you go to:

```
IIS APPPOOL\Dapper_CRUD
  → Properties
  → User Mapping
  → Dapper
```

SQL Server may show the **database user name** as:

```
Dapper_CRUD
```

That's normal.

The relationship is:

```
Windows/IIS Identity
  ↓
IIS APPPOOL\Dapper_CRUD
  ↓
SQL Server Login
  ↓
Dapper database
  ↓
Database User = Dapper_CRUD
```

So you should **not change** `Dapper\_CRUD` to `IIS APPPOOL\Dapper\_CRUD` in the User Mapping user field.

What you should verify now

In **Login Properties → User Mapping**:

Database	Map

☒ Dapper	

Then at the bottom:

```
Database role membership for: Dapper

☒ public
☒ db_datareader
```

☒ db_datawriter

If those are selected, click **OK**.

Then we can move to the next step: **testing your API through IIS**.

One important distinction:

* **Login name:** IIS APPPOOL\Dapper_CRUD ✓

* **Database user:** Dapper_CRUD ✓

They are supposed to look different.

Trying to Recycle Where it is?

That's likely because you're looking at the **website** (`Sites → Dapper\_CRUD`), not the **Application Pool**.

Go here:

In IIS Manager, on the **left panel**:

```
LAPTOP-ED55U690
├── Application Pools ← click this
├── Sites
│   └── Dapper_CRUD
```

Click **Application Pools**.

You should see:

Dapper_CRUD

in the middle panel.

Then:

1. **Right-click `Dapper\_CRUD`**

2. You should see:

- * Start
- * Stop
- * **Recycle**
- * Advanced Settings
- * Basic Settings
- * Remove

Click:

Recycle

If you still don't see Recycle

Select `Dapper_CRUD` once.

Look at the **right-hand Actions panel**. Under **Application Pool Tasks**, you should see:

```
Stop
Recycle
Advanced Settings...
Basic Settings...
```

Click **Recycle** there.

One important point

You don't actually need to recycle it if you **just created the Application Pool and Website** and haven't changed anything after starting it. Recycling is mainly useful after configuration changes.

So if you cannot find it, **don't get stuck here**. We can proceed directly to testing:

```
http://localhost:8080/api/Employee
```

That's the more important next step.

Getting the URLs to Test

Based on your controller, your IIS base URL is:

```
http://localhost:8080
```

and because you have:

```
[Route("api/[controller]")]
public class EmployeeController : ControllerBase
```

`[controller]` becomes `Employee`.

So your APIs are:

Method	IIS URL	Purpose
GET	http://localhost:8080/api/Employee	Get all employees
GET	http://localhost:8080/api/Employee/1	Get employee with ID 1
POST	http://localhost:8080/api/Employee	Add employee
PUT	http://localhost:8080/api/Employee/1	Update employee with ID 1
DELETE	http://localhost:8080/api/Employee/1	Delete employee with ID 1

1. GET all employees

Open this in your browser:

```
http://localhost:8080/api/Employee
```

Your method:

```
[HttpGet]  
public async Task<IActionResult> GetEmployees()
```

If successful, you'll get something like:

```
[  
  {  
    "employeeId": 1,  
    "name": "John",  
    "salary": 50000  
  },  
  {  
    "employeeId": 2,  
    "name": "Rahul",  
    "salary": 60000  
  }  
]
```

This is the **first API I'd test through IIS**.

2. GET employee by ID

For employee ID 1:

```
http://localhost:8080/api/Employee/1
```

This maps to:

```
[HttpGet("{id}")]  
public async Task<IActionResult> GetEmployeeById(int id)
```

If ID 1 exists:

```
{  
  "employeeId": 1,  
  "name": "John",  
  "salary": 50000  
}
```

If it doesn't:

```
Employee Not Found
```

with HTTP status:

```
404 Not Found
```

3. POST — Add employee

Use **Postman**, because this requires a request body.

```
POST
http://localhost:8080/api/Employee
```

Headers:

```
Content-Type: application/json
```

Body → **raw** → **JSON**:

```
{
  "employeeId": 0,
  "name": "Test Employee",
  "salary": 50000
}
```

The exact JSON property names should match your `Employee` model.

Your controller:

```
[HttpPost]
public async Task<IActionResult> AddNewEmp(Employee e)
```

returns:

```
{
  "message": "Emp added Successfully"
}
```

4. PUT — Update employee

For employee ID `1`:

```
PUT
http://localhost:8080/api/Employee/1
```

Body:

```
{
  "employeeId": 0,
  "name": "Updated Employee",
  "salary": 70000
}
```

Your controller does:

```
e.EmployeeId = id;
```

So even if you send:

```
"employeeId": 0
```

the controller changes it to:

```
1
```

because the URL contains:

```
/Employee/1
```

Response:

```
{  
  "message": "Employee Updated Successfully"  
}
```

5. DELETE — Delete employee

For employee ID 1:

```
DELETE  
http://localhost:8080/api/Employee/1
```

No request body is required.

Your controller:

```
[HttpDelete("{id}")]  
public async Task<IActionResult> DeleteEmployee(int id)
```

If successful:

```
{  
  "message": "Employee deleted successfully"  
}
```

What I recommend you test RIGHT NOW

Don't test POST/PUT/DELETE yet.

First test:

```
http://localhost:8080/api/Employee
```

in your browser.

Possible results

1. You get **JSON data** → Excellent. Your IIS deployment is working:

```
Browser
↓
IIS :8080
↓
Dapper_CRUD App Pool
↓
ASP.NET Core
↓
EmployeeController
↓
Service
↓
Dapper
↓
SQL Server
```

2. You get **`404 Not Found`** → We need to check IIS routing/site configuration.

3. You get **`500.30`** → ASP.NET Core application failed during startup.

4. You get **`500.31`** → Required .NET runtime/framework issue.

5. You get **`500 Internal Server Error`** → Could be application/database/configuration related.

6. You get **`502.5`** → IIS couldn't start/communicate with the ASP.NET Core process.

So open this exact URL now:

```
http://localhost:8080/api/Employee
```

and tell me exactly what response/error you get.

ERROR 500 - # This page isn't working

localhost is currently unable to handle this request.

HTTP ERROR 500

That is useful: **IIS is receiving the request, but your ASP.NET Core application is failing somewhere behind IIS.** Don't change random settings yet.

For your setup, the most likely causes are:

1. Dapper_CRUD IIS application pool identity cannot access SQL Server.
2. The published appsettings.json/connection string is wrong.
3. The .NET 8 Hosting Bundle/runtime isn't correctly installed.
4. Your application is throwing an exception during startup or while executing `GetEmployees()`.

Let's identify the exact error.

Step 1 — Check Windows Event Viewer

Press:

```
Win + R
```

Type:

```
eventvwr
```

Press Enter.

Go to:

```
Windows Logs  
↓  
Application
```

Look for a recent **Error** at the time you tried:

```
http://localhost:8080/api/Employee
```

Look for entries mentioning:

```
ASP.NET Core  
IIS  
IIS AspNetCore Module  
Dapper_CRUD  
.NET Runtime
```

If you find one, send me the **error message/details**.

Step 2 — Enable ASP.NET Core stdout logging

This is even more useful.

Go to your published folder:

```
C:\Publish\EmployeeManagement
```

Open:

```
web.config
```

You'll probably have something similar to:

```
<aspNetCore processPath="dotnet"  
  arguments=". \Dapper_CRUD.dll"  
  stdoutLogEnabled="false"  
  stdoutLogFile=".\logs\stdout"  
  hostingModel="inprocess" />
```

Change:

```
stdoutLogEnabled="false"
```

to:

```
stdoutLogEnabled="true"
```

So:

```
<aspNetCore processPath="dotnet"
  arguments=".\\Dapper_CRUD.dll"
  stdoutLogEnabled="true"
  stdoutLogFile=".\logs\\stdout"
  hostingModel="inprocess" />
```

Then create a logs folder

Inside:

```
C:\\Publish\\EmployeeManagement
```

create:

```
logs
```

So:

```
C:\\Publish\\EmployeeManagement
├──
├── Dapper_CRUD.dll
├── web.config
├── appsettings.json
├── ...
└── logs
```

Then make sure the IIS identity:

```
IIS APPPOOL\\Dapper_CRUD
```

has **Write** permission on that logs folder.

Step 3 — Restart the IIS site

In IIS:

```
Sites
  ↓
Dapper_CRUD
```

Right-click:

```
Manage Website
  ↓
```

```
Restart
```

Then open again:

```
http://localhost:8080/api/Employee
```

Step 4 — Check the log

Go back to:

```
C:\Publish\EmployeeManagement\logs
```

You should hopefully get a file such as:

```
stdout_....log
```

Open that file.

It should tell us the **actual .NET exception**.

For example, if the problem is SQL Server permissions, you may see something similar to:

```
Login failed for user 'IIS APPPOOL\Dapper_CRUD'
```

If the Hosting Bundle is the problem, you'll see a runtime-related error.

If your application has a configuration problem, you'll see that instead.

One important thing

You **already created**:

```
IIS APPPOOL\Dapper_CRUD
```

in SQL Server, so that's good.

But make sure in **Login Properties → User Mapping** you have:

```
Dapper ☒
```

and:

```
public ☒  
db_datareader ☒  
db_datawriter ☒
```

Then click **OK**.

Don't modify your C# controller

Your controller is not the first thing I'd change. A generic IIS 500 is an **environment/deployment error until proven otherwise**.

Do Step 2 and reproduce the error. Then open the generated `stdout\_\*.log` and paste its contents here. That will let us diagnose the actual failure instead of guessing.

Microsoft.Data.SqlClient.SqlException (0x80131904): The EXECUTE permission was denied on the object 'sp_GetEmployeeById', database 'Dapper', schema 'dbo'. at Microsoft.Data.SqlClient.SqlCommand.

Now we have the **exact problem**. Your IIS and ASP.NET Core deployment are actually working. The failure is specifically a **SQL Server permission issue**.

The most important line in your logs is:

```
The EXECUTE permission was denied on the object 'sp_GetEmployee',  
database 'Dapper', schema 'dbo'.
```

And the first log shows the same problem for:

```
sp_GetEmployeeById
```

What is happening

Your request flow is:

```
Browser  
↓  
http://localhost:8080/api/Employee  
↓  
IIS  
↓  
Dapper_CRUD Application Pool  
↓  
ASP.NET Core application  
↓  
EmployeeController  
↓  
EmpService  
↓  
SQL Server  
↓  
Dapper database  
↓  
sp_GetEmployee ← X PERMISSION DENIED
```

So we can already confirm:

✔ **IIS is working**

Your log says:

```
Application started.  
Hosting environment: Production
```

```
Content root path: D:\IIS-Publish
```

That means your published application successfully started under IIS.

✓ASP.NET Core is working

The request reached:

```
EmployeeController.GetEmployees()
```

So routing and your controller are working.

✓Connection to SQL Server is working

This is important.

If the connection string itself were completely wrong, you'd get something like:

```
Login failed
Cannot open database
A network-related error occurred
```

Instead, SQL Server is telling you:

> I know who this user is, I know which database you're accessing, but this user doesn't have permission to execute this stored procedure.

✗SQL Server stored-procedure permissions are missing

Your IIS application is using:

```
IIS APPPOOL\Dapper_CRUD
```

to connect to SQL Server because your connection string uses:

```
Trusted_Connection=True
```

Therefore SQL Server sees the application as:

```
IIS APPPOOL\Dapper_CRUD
```

You created that login, which was correct.

But **creating the login/user alone isn't enough**.

That user needs permission to execute your stored procedures.

The exact problem

Your application calls:

```
sp_GetEmployee
```

but SQL Server responds:

```
Error Number: 229  
EXECUTE permission was denied
```

You also have:

```
sp_GetEmployeeById
```

with the same problem.

And because your CRUD application probably has other stored procedures, you will likely encounter the same error for:

```
sp_AddEmployee  
sp_UpdateEmployee  
sp_DeleteEmployee
```

or whatever your actual procedure names are.

The easiest solution for your practice project

Since this is your **Employee Management/Dapper CRUD practice application**, you can give your database user permission to execute stored procedures.

Open **SSMS**.

Select:

```
Dapper
```

and click:

New Query

Run:

```
USE Dapper;  
GO  
  
GRANT EXECUTE TO [Dapper_CRUD];  
GO
```

This grants the `Dapper_CRUD` database user permission to execute stored procedures in the database.

Then test

Go to:

```
http://localhost:8080/api/Employee
```

You should now get your employee data instead of HTTP 500.

But let's first verify the user

Run this in SSMS:

```
USE Dapper;
GO

SELECT
    name,
    type_desc
FROM sys.database_principals
WHERE name = 'Dapper_CRUD';
```

You should get something like:

name	type_desc
-----	-----
Dapper_CRUD	WINDOWS_USER

That confirms your IIS application pool identity has a database user.

Then check the permissions

Run:

```
USE Dapper;
GO

SELECT
    dp.name AS UserName,
    p.permission_name,
    p.state_desc
FROM sys.database_principals dp
LEFT JOIN sys.database_permissions p
    ON dp.principal_id = p.grantee_principal_id
WHERE dp.name = 'Dapper_CRUD';
```

You may currently find that there is **no EXECUTE permission**.

That's why your application fails.

Recommended fix

For your current development/practice deployment:

```
USE Dapper;  
GO  
  
GRANT EXECUTE TO [Dapper_CRUD];  
GO
```

This is better than giving the IIS account:

```
db_owner
```

Don't do this:

```
ALTER ROLE db_owner ADD MEMBER [Dapper_CRUD];
```

It would probably make the application work, but it gives the IIS application far more database privileges than it needs.

For a real application, **least privilege** is the better approach.

Why db_datareader and db_datawriter weren't enough

This is an important concept for you as a .NET developer.

You may have selected:

```
☒ db_datareader  
☒ db_datawriter
```

Those roles basically allow things like:

```
SELECT  
INSERT  
UPDATE  
DELETE
```

on tables.

But your application isn't directly doing:

```
SELECT * FROM Employee
```

Your Dapper code is doing something like:

```
connection.QueryAsync<Employee>(  
    "sp_GetEmployee",  
    CommandType.StoredProcedure  
);
```

That means SQL Server requires:

```
EXECUTE
```


permission on the stored procedure.

So:

```
db_datareader
+
db_datawriter
```

does **not automatically mean**

```
EXECUTE stored procedure
```

That's exactly what you're running into.

Your second log confirms this

Your second log says:

```
The EXECUTE permission was denied on the object 'sp_GetEmployee'
```

and the stack trace shows:

```
Dapper_CRUD.Service.EmpService.GetEmployees()
```

then:

```
Dapper_CRUD.Controllers.EmployeeController.GetEmployees()
```

So your request:

```
GET /api/Employee
```

is reaching this:

```
[HttpGet]
public async Task<IActionResult> GetEmployees()
{
    var data = await service.GetEmployees();
    return Ok(data);
}
```

Then:

```
EmployeeController
    ↓
EmpService.GetEmployees()
    ↓
Dapper
    ↓
sp_GetEmployee
    ↓
❌Permission denied
```

That's why the browser gets:

```
HTTP ERROR 500
```

What about this warning?

You also have:

```
Failed to determine the https port for redirect.
```

Don't confuse this with the main error.

It is a **separate warning**.

Your actual fatal error is:

```
Error Number: 229  
The EXECUTE permission was denied on the object 'sp_GetEmployee'
```

Fix the SQL permission first.

Do this now

1. Open SSMS

2. Run:

```
USE Dapper;  
GO  
  
GRANT EXECUTE TO [Dapper_CRUD];  
GO
```

3. Recycle your IIS application pool

```
IIS  
→ Application Pools  
→ Dapper_CRUD  
→ Right-click  
→ Recycle
```

4. Test:

```
http://localhost:8080/api/Employee
```

5. Then test:

```
http://localhost:8080/api/Employee/1
```

The second one is important because your logs showed that `sp_GetEmployeeById` also lacks EXECUTE permission.

One more thing

After you grant:

```
GRANT EXECUTE TO [Dapper_CRUD];
```

you don't need to republish your .NET application.

You don't need to change:

- * Controller ✗
- * Service ✗
- * Dapper code ✗
- * `appsettings.json` ✗
- * IIS website ✗
- * Application Pool ✗
- * Hosting Bundle ✗

The application is already reaching SQL Server successfully.

This is purely a **database authorization/permission issue**.

Once `GRANT EXECUTE` is applied, your GET API should move past this error.

IT IS RETURNING EMPTY

If this query:

```
USE Dapper;
GO

SELECT
    name,
    type_desc
FROM sys.database_principals
WHERE name = 'Dapper_CRUD';
```

is returning **empty**, then we have found the next issue.

It means the login:

```
IIS APPPOOL\Dapper_CRUD
```

exists at the **SQL Server level**, but there is currently **no corresponding database user inside the `Dapper` database**.

That's why granting permissions to [Dapper_CRUD] may not work.

Fix it

In SSMS, run this:

```
USE Dapper;  
GO  
  
CREATE USER [Dapper_CRUD] FOR LOGIN [IIS APPPOOL\Dapper_CRUD];  
GO
```

Then verify:

```
SELECT  
    name,  
    type_desc  
FROM sys.database_principals  
WHERE name = 'Dapper_CRUD';
```

You should now see:

```
Dapper_CRUD    WINDOWS_USER
```

Then grant the required permissions

Since your application uses stored procedures:

```
USE Dapper;  
GO  
  
GRANT EXECUTE TO [Dapper_CRUD];  
GO
```

For your current CRUD application, you can also explicitly grant the procedure permissions instead of granting EXECUTE across the entire database:

```
GRANT EXECUTE ON dbo.sp_GetEmployee TO [Dapper_CRUD];  
GRANT EXECUTE ON dbo.sp_GetEmployeeById TO [Dapper_CRUD];
```

And similarly for your Add/Update/Delete procedures once we confirm their names.

Then test

Recycle:

```
IIS  
→ Application Pools  
→ Dapper_CRUD  
→ Recycle
```

Then:

```
http://localhost:8080/api/Employee
```

One important distinction

You currently have:

```
SQL Server
├── Security
│   ├── Logins
│       └── IIS APPPOOL\Dapper_CRUD ← Login exists ✓
```

But you need:

```
Dapper database
├── Security
│   ├── Users
│       └── Dapper_CRUD ← Database user ✗currently missing
```

The **login** allows the IIS identity to authenticate to SQL Server.

The **database user** gives that login access to a specific database.

That's the missing piece here.

Msg 15151, Level 16, State 1, Line 15

Cannot find the user 'Dapper', because it does not exist or you do not have permission. The login already has an account under a different user name.

That error is actually **good news** because it tells us exactly what happened.

The key part is:

> **The login already has an account under a different user name.**

This means the SQL Server login:

```
IIS APPPOOL\Dapper_CRUD
```

already has a user inside the `Dapper` database, but that database user is **not named `Dapper\_CRUD`**.

This matches what you saw earlier in SSMS where the database user was appearing as **`Dapper`**.

So **do not run `CREATE USER` again**. We need to find the existing database user and grant it EXECUTE permission.

Step 1 — Find the actual database user

In SSMS, open **New Query** and run:

```
USE Dapper;
GO
```

```
SELECT
    dp.name AS DatabaseUser,
    sp.name AS ServerLogin
FROM sys.database_principals dp
INNER JOIN sys.server_principals sp
    ON dp.sid = sp.sid
WHERE sp.name = 'IIS APPPOOL\Dapper_CRUD';
```

You should get something similar to:

DatabaseUser	ServerLogin
Dapper	IIS APPPOOL\Dapper_CRUD

If it shows:

```
Dapper
```

then **Dapper** is the database username that we need to grant permissions to.

Step 2 — Grant EXECUTE to the existing user

If the result is:

```
DatabaseUser = Dapper
```

run:

```
USE Dapper;
GO

GRANT EXECUTE TO [Dapper];
GO
```

This is the important correction.

Previously we were trying to grant permission to:

```
[Dapper_CRUD]
```

but your database user is apparently:

```
[Dapper]
```

Step 3 — Verify the permission

Run:

```
USE Dapper;
```

```
GO

SELECT
    dp.name AS DatabaseUser,
    p.permission_name,
    p.state_desc
FROM sys.database_principals dp
LEFT JOIN sys.database_permissions p
    ON dp.principal_id = p.grantee_principal_id
WHERE dp.name = 'Dapper';
```

You should see something like:

DatabaseUser	permission_name	state_desc
-----	-----	-----
Dapper	EXECUTE	GRANT

Step 4 — Check the stored procedures

Your logs have already identified these two:

```
dbo.sp_GetEmployee
dbo.sp_GetEmployeeById
```

You can explicitly grant them too:

```
USE Dapper;
GO

GRANT EXECUTE ON dbo.sp_GetEmployee TO [Dapper];
GRANT EXECUTE ON dbo.sp_GetEmployeeById TO [Dapper];
GO
```

For your CRUD application, you'll also need the Add, Update and Delete procedures.

First find their names:

```
USE Dapper;
GO

SELECT name
FROM sys.procedures
ORDER BY name;
```

You will get your stored procedure list.

Why this happened

Your setup is currently something like this:

```
SQL Server
├── Login
│   └── IIS APPPOOL\Dapper_CRUD
└── Database: Dapper
    ├── User
    │   └── Dapper
```

The names **don't have to be the same**.

That's perfectly valid.

The important relationship is:

```
IIS APPPOOL\Dapper_CRUD
    ↓
SQL Server Login
    ↓
Database User: Dapper
    ↓
Dapper Database
    ↓
EXECUTE permission
    ↓
Stored Procedures
```

Your application is already reaching SQL Server. The failure was simply:

```
IIS APPPOOL\Dapper_CRUD
    ↓
Database User: Dapper
    ↓
✗ No EXECUTE permission
    ↓
sp_GetEmployee
    ↓
HTTP 500
```

Do this first

Run **only this query** and show me the result:

```
USE Dapper;
GO

SELECT
```



```

    dp.name AS DatabaseUser,
    sp.name AS ServerLogin
FROM sys.database_principals dp
INNER JOIN sys.server_principals sp
    ON dp.sid = sp.sid
WHERE sp.name = 'IIS APPPOOL\Dapper_CRUD';

```

Once we see the exact DatabaseUser, I'll tell you the precise permission command to run.

DatabaseUser	ServerLogin
IIS APPPOOL\Dapper_CRUD	IIS APPPOOL\Dapper_CRUD

Perfect. This confirms the setup clearly.

Your result:

DatabaseUser	ServerLogin
IIS APPPOOL\Dapper_CRUD	IIS APPPOOL\Dapper_CRUD

means:

```

IIS Application Pool
    ↓
Dapper_CRUD
    ↓
SQL Server Login
    ↓
IIS APPPOOL\Dapper_CRUD
    ↓
Dapper Database User
    ↓
Dapper database

```

So **the database user exists and is correctly mapped to the IIS login**. The previous CREATE USER step is not needed.

Now fix the actual problem

Your logs say:

```
EXECUTE permission was denied on the object 'sp_GetEmployee'
```

and:

```
EXECUTE permission was denied on the object 'sp_GetEmployeeById'
```

Therefore, give this database user permission to execute your stored procedures.

Run this in SSMS:

```

USE Dapper;
GO

```

```
GRANT EXECUTE ON dbo.sp_GetEmployee TO [IIS APPPOOL\Dapper_CRUD];
GRANT EXECUTE ON dbo.sp_GetEmployeeById TO [IIS APPPOOL\Dapper_CRUD];
GO
```

Verify

Run:

```
USE Dapper;
GO

SELECT
    dp.name AS DatabaseUser,
    p.permission_name,
    p.state_desc
FROM sys.database_principals dp
INNER JOIN sys.database_permissions p
    ON dp.principal_id = p.grantee_principal_id
WHERE dp.name = 'IIS APPPOOL\Dapper_CRUD';
```

You should see:

DatabaseUser	permission_name	state_desc
-----	-----	-----
IIS APPPOOL\Dapper_CRUD	EXECUTE	GRANT
IIS APPPOOL\Dapper_CRUD	EXECUTE	GRANT

One will be for `sp_GetEmployee` and the other for `sp_GetEmployeeById`.

But your CRUD has more procedures

Your application has:

- * Get all employees
- * Get employee by ID
- * Add employee
- * Update employee
- * Delete employee

So you will eventually need EXECUTE permission for the other three stored procedures as well.

First run:

```
USE Dapper;
GO

SELECT name
```

```
FROM sys.procedures
ORDER BY name;
```

You can paste the result here.

I'll identify **exactly which three procedures correspond to Add, Update and Delete** and give you the complete permission script.

For now, don't change your C# code or appsettings.json.

Your current evidence shows:

IIS → ASP.NET Core → SQL Server → Dapper database is working.

The remaining issue is **stored-procedure authorization**.

name

sp_AddEmp

sp_AddManager

sp_DeleteEmployee

sp_GetEmployee

sp_GetEmployeeById

sp_GetManagerById

sp_GetManagers

sp_UpdateEmployee

sp_UpdatEmployees

Good. Now we have the complete stored-procedure list.

For your **Employee Management System**, the relevant procedures are:

Operation	Stored Procedure
Add Employee	sp_AddEmp
Get All Employees	sp_GetEmployee
Get Employee by ID	sp_GetEmployeeById
Update Employee	sp_UpdateEmployee or possibly sp_UpdatEmployees
Delete Employee	sp_DeleteEmployee

The manager procedures are not relevant to the controller you showed.

1. Give IIS permission for all Employee procedures

Run this:

```
USE Dapper;
GO
```

```
GRANT EXECUTE ON dbo.sp_AddEmp TO [IIS APPPOOL\Dapper_CRUD];
GRANT EXECUTE ON dbo.sp_GetEmployee TO [IIS APPPOOL\Dapper_CRUD];
GRANT EXECUTE ON dbo.sp_GetEmployeeById TO [IIS APPPOOL\Dapper_CRUD];
GRANT EXECUTE ON dbo.sp_DeleteEmployee TO [IIS APPPOOL\Dapper_CRUD];
GRANT EXECUTE ON dbo.sp_UpdateEmployee TO [IIS APPPOOL\Dapper_CRUD];
GO
```

2. What about sp_UpdatEmployees?

You have both:

```
sp_UpdateEmployee
sp_UpdatEmployees
```

There appears to be a naming difference:

```
sp_UpdateEmployee    ← likely the procedure your current service uses
sp_UpdatEmployees    ← another procedure
```

Do not grant permission to `sp\_UpdatEmployees` yet unless your `EmpService.cs` actually calls it.

Your controller calls:

```
await service.UpdateEmployee(e);
```

The actual stored procedure being used is determined inside `EmpService.UpdateEmployee()`.

3. You can also grant EXECUTE at database level

Since this is your local Employee Management System and you're deploying the whole API, the simpler approach is:

```
USE Dapper;
GO

GRANT EXECUTE TO [IIS APPPOOL\Dapper_CRUD];
GO
```

This gives the IIS application-pool user permission to execute **all stored procedures in the `Dapper` database**, including future procedures.

For your current local IIS setup, I would use this approach rather than manually maintaining permissions for every stored procedure.

It does **not** give the application full database ownership. It specifically grants the ability to execute stored procedures.

4. Recommended approach for your current setup

Run:

```
USE Dapper;  
GO  
  
GRANT EXECUTE TO [IIS APPPOOL\Dapper_CRUD];  
GO
```

Then verify:

```
SELECT  
    dp.name AS DatabaseUser,  
    dp.type_desc,  
    p.permission_name,  
    p.state_desc  
FROM sys.database_principals dp  
LEFT JOIN sys.database_permissions p  
    ON dp.principal_id = p.grantee_principal_id  
WHERE dp.name = 'IIS APPPOOL\Dapper_CRUD';
```

However, because `GRANT EXECUTE TO user` is a database-level permission, you may see:

```
EXECUTE    GRANT
```

with the permission applying to the database rather than each individual procedure.

Then restart/recycle IIS

In **IIS Manager**:

```
Application Pools  
↓  
Dapper_CRUD  
↓  
Right-click  
↓  
Recycle
```

Then test:

```
http://localhost/api/Employee
```

or whatever port/binding you configured.

Important

Your previous logs showed:

```
EXECUTE permission was denied on the object 'sp_GetEmployee'
```

So after granting:

```
GRANT EXECUTE TO [IIS APPPOOL\Dapper_CRUD];
```

that specific error should be resolved.

If you then get another error, **don't start changing random IIS settings**. Send me the new error/log. We can work through the deployment layer by layer.

Application activation status on IIS Server

Yes — from your screenshot, **`Dapper\_CRUD` is currently a Site, but it has no Application under it**. That's why the Applications pane is empty.

You need to convert/add your published folder as an **IIS Application** and assign your `Dapper_CRUD` application pool to it.

Do this step-by-step

1. Right-click Dapper_CRUD

In the left panel:

```
Sites
└─ Dapper_CRUD
```

Right-click **Dapper_CRUD**.

You should see:

Add Application...

Click it.

2. Configure the application

A window will appear with fields similar to:

```
Alias:
Application pool:
Physical path:
```

Set them like this:

Alias:

```
api
```

You can also use:

```
Dapper_CRUD
```

But since this is your API, I recommend:

```
api
```

Application pool:

Click **Select...**

Choose:

```
Dapper_CRUD
```

This is the application pool you created earlier.

Physical path:

Set it to your published folder:

```
D:\IIS-Publish
```

Based on your earlier logs, this is definitely your publish folder:

```
Content root path: D:\IIS-Publish
```

So your configuration should effectively be:

```
Alias:          api
Application pool: Dapper_CRUD
Physical path:  D:\IIS-Publish
```

Click **OK**.

3. What you should see afterward

Your IIS tree should become approximately:

```
Sites
├── Dapper_CRUD
│   └── api
```

And when you click `api`, the Applications pane should show the application.

The important distinction is:

```
Site
  ↓
Application
  ↓
Application Pool
  ↓
Published .NET application
```

Your current screenshot has the **Site**, but the **Application** hasn't been added yet.

4. Then test the API

If your Dapper_CRUD site is configured on, for example, port 8080, your API URL becomes:

```
http://localhost:8080/api/Employee
```

If you used a different alias, the URL changes accordingly.

Do not create another application pool. You already have the correct one:

```
Dapper_CRUD
```

After adding the application, if you send me a screenshot of the **Add Application** window before clicking OK, I can tell you exactly what to put in each field.

Here there is a option Pass through authentication

(1) Connect as

There's is Specific User :-

and Application user

(2) Test Settings

Tell is it necessary

What to select

No, **you do not need to use "Specific User"** here.

Based on your screenshot and your current IIS setup, use:

What to select

Under **Pass-through authentication** → **Connect as...**

Select:

☒ **Application user (pass-through authentication)**

Do **not** select:

☐ **Specific user**

Then click **OK**.

What this means

Your setup is:

```
IIS Site
  ↓
Dapper_CRUD Application Pool
  ↓
Application Pool Identity:
IIS APPPOOL\Dapper_CRUD
```



```
↓  
D:\IIS-Publish  
↓  
ASP.NET Core API  
↓  
SQL Server → Dapper database
```

You have already created the SQL Server login:

```
IIS APPPOOL\Dapper_CRUD
```

and the database user is also:

```
IIS APPPOOL\Dapper_CRUD
```

So we **want IIS to run the application using the Application Pool Identity**, rather than introducing another Windows account.

What about "Test Settings"?

You **can click Test Settings**, but it is not the thing that fixes your current SQL permission problem.

Your previous error was:

```
The EXECUTE permission was denied on the object 'sp_GetEmployee'
```

That means:

> IIS successfully reached your application → application successfully reached SQL Server → SQL Server recognized the IIS identity → but that identity does not have EXECUTE permission on your stored procedures.

So the **main problem is SQL permissions**, not Pass-through Authentication.

Do this now

In the window shown in your screenshot:

1. Keep **Application user (pass-through authentication)** selected.
2. Click **OK** on the small **Connect As** window.
3. Back on the **Add Application** window, click **OK**.
4. IIS should create the application under Dapper_CRUD.

Then we will verify the application is using:

```
Application Pool = Dapper_CRUD
```

After that, **do not change the connection settings again**.

The next important step is to grant:

EXECUTE

permission on these stored procedures to:

IIS APPPOOL\Dapper_CRUD

because your logs clearly show that this is the actual cause of HTTP 500.

We should grant permission to all the procedures your API uses, not just `sp_GetEmployee`.

From the start when we started to deploy the application on IIS give me the steps for the same in detail and in simple words so that others might understand

Also include the commands we use to download the application and every other detail as I have to document it for the company employees

Absolutely. Below is a **company-documentation-style deployment procedure** based on the IIS deployment we have been doing for your **Dapper CRUD ASP.NET Core Web API**.

I have written it so that another developer who has never deployed an ASP.NET Core API to IIS can follow it from beginning to end.

> **Example application used in this document:** Dapper_CRUD

> **Framework:** .NET 8

> **Database:** SQL Server / SQL Express

> **Database:** Dapper

> **IIS Application Pool:** Dapper_CRUD

> **Published application folder:** D:\IIS-Publish

ASP.NET Core Web API Deployment on IIS

1. Purpose

This document explains how to deploy an **ASP.NET Core Web API application** on **Microsoft IIS (Internet Information Services)**.

The deployment process consists of:

```
Source Code
↓
Download / Clone Application
↓
Restore NuGet Packages
↓
Build Application
↓
Publish Application
```

```
↓  
Install .NET Hosting Bundle  
↓  
Install / Configure IIS  
↓  
Create IIS Application Pool  
↓  
Create IIS Website/Application  
↓  
Configure SQL Server Permissions  
↓  
Test API
```

2. Prerequisites

Before starting, make sure the server/development machine has:

Software

- * Windows
- * IIS
- * .NET SDK required by the application
- * ASP.NET Core Hosting Bundle
- * SQL Server / SQL Express
- * SQL Server Management Studio (SSMS)
- * Git, if source code is stored in Git
- * Web browser
- * Administrator access

For our application:

```
.NET: 8.0  
Database: SQL Server Express  
Database Name: Dapper
```

3. Check the .NET Version Used by the Application

First open the application's `.csproj` file.

For example:

```
<TargetFramework>net8.0</TargetFramework>
```

This tells us that the application is built using:

```
.NET 8
```

This is important because IIS must have the appropriate ASP.NET Core runtime/Hosting Bundle installed.

4. Check Installed .NET Versions

Open **Command Prompt** or **PowerShell** and run:

```
dotnet --version
```

This shows the currently selected SDK.

Example:

```
8.0.xxx
```

To see all installed SDKs:

```
dotnet --list-sdks
```

To see all installed runtimes:

```
dotnet --list-runtimes
```

For example:

```
Microsoft.AspNetCore.App 8.0.30  
Microsoft.NETCore.App 8.0.30
```

5. Download the Application Source Code

If the application is stored in GitHub/Git, first install Git.

Check whether Git is installed:

```
git --version
```

Example:

```
git version 2.x.x
```

If Git is not installed, install Git on the machine.

5.1 Clone the Application

Go to the folder where you want to keep the source code.

Example:

```
cd D:\
```

Create a source-code folder:

```
mkdir Projects
```

Go inside it:

```
cd D:\Projects
```

Clone the repository:

```
git clone <REPOSITORY-URL>
```

For example:

```
git clone https://github.com/company/Dapper-API-React.git
```

Important: Replace the URL with your company's actual repository URL.

After cloning:

```
cd Dapper-API-React
```

6. Verify the Project

Find the `.csproj` file.

For example:

```
D:\Projects\Dapper-API-React\Dapper_CRUD\Dapper_CRUD.csproj
```

Move into the project directory:

```
cd D:\Projects\Dapper-API-React\Dapper_CRUD
```

Check the files:

```
dir
```

You should see things such as:

```
Controllers
Models
Service
Program.cs
appsettings.json
Dapper_CRUD.csproj
```

7. Restore NuGet Packages

Run:

```
dotnet restore
```

What does this do?

The application may use packages such as:

- * Dapper
- * Microsoft.Data.SqlClient
- * ASP.NET Core packages
- * Swagger/OpenAPI packages

`dotnet restore` downloads/restores the required NuGet packages.

Think of it as:

```
Project files
  ↓
dotnet restore
  ↓
Required NuGet packages downloaded
```

8. Build the Application

Run:

```
dotnet build
```

If everything is correct, you should see:

```
Build succeeded.
```

If there are compilation errors, fix them before continuing.

9. Configure the Database Connection

Open:

```
appsettings.json
```

For our application, the connection string was:

```
{
  "ConnectionStrings": {
    "DbConn":
    "Server=localhost\\SQLEXPRESS;Database=Dapper;Trusted_Connection=True;TrustServerCertificate=True;"
  }
}
```

Explanation

Server=localhost\SQLEXPRESS

SQL Server Express is running on the same machine.

Database=Dapper

The application connects to the Dapper database.

Trusted_Connection=True

Windows Authentication is being used.

TrustServerCertificate=True

The SQL Server certificate is trusted for this connection.

10. Verify SQL Server Database

Open SSMS.

Connect to:

localhost\SQLEXPRESS

Go to:

Databases

↓

Dapper

Make sure the database exists.

11. Verify Required Tables

Inside:

Dapper

↓

Tables

Verify the required tables exist.

For example:

```
Employees  
Managers
```

depending on your application.

12. Verify Stored Procedures

Go to:

```
Dapper  
↓  
Programmability  
↓  
Stored Procedures
```

Our application contained procedures such as:

```
sp_AddEmp  
sp_AddManager  
sp_DeleteEmployee  
sp_GetEmployee  
sp_GetEmployeeById  
sp_GetManagerById  
sp_GetManagers  
sp_UpdateEmployee  
sp_UpdatEmployees
```

The exact list depends on the application.

13. Test Stored Procedures in SSMS

Before IIS deployment, verify that the procedures themselves work.

For example:

```
EXEC sp_GetEmployee;
```

For a particular employee:

```
EXEC sp_GetEmployeeById 1;
```

For the other procedures, use their required parameters.

This separates:


```
SQL problem
```

from:

```
IIS/application problem
```

14. Publish the Application

This is one of the most important steps.

From the project directory:

```
cd D:\Projects\Dapper-API-React\Dapper_CRUD
```

Run:

```
dotnet publish -c Release
```

The default publish folder will normally be:

```
bin\Release\net8.0\publish
```

So, for example:

```
D:\Projects\Dapper-API-React\Dapper_CRUD\bin\Release\net8.0\publish
```

15. Publish Directly to IIS Folder

Instead of manually copying the files, we can publish directly to:

```
D:\IIS-Publish
```

First create the folder:

```
mkdir D:\IIS-Publish
```

Then run:

```
dotnet publish -c Release -o D:\IIS-Publish
```

Now the compiled application will be placed directly into:

```
D:\IIS-Publish
```

You should see files such as:

```
D:\IIS-Publish
|
├─ Dapper_CRUD.dll
├─ Dapper_CRUD.deps.json
```

```
|— Dapper_CRUD.runtimeconfig.json
|— appsettings.json
|— web.config
|— other DLLs
```

16. Why dotnet publish Is Required

Do not simply copy the source-code folder to IIS.

For example, this is **not the proper IIS deployment approach**:

```
| D:\Projects\Dapper-API-React
```

Instead, IIS should point to the **published output**:

```
| D:\IIS-Publish
```

The publishing process prepares the application for deployment.

17. Install ASP.NET Core Hosting Bundle

This is required for ASP.NET Core applications running behind IIS.

The Hosting Bundle installs the components IIS needs to host ASP.NET Core applications.

For a .NET 8 application, install the appropriate:

```
| ASP.NET Core Hosting Bundle for .NET 8
```

In our environment we had:

```
| Microsoft.AspNetCore.App 8.0.30
```

and the .NET 8 Hosting Bundle was installed.

Important distinction

Do not document:

> "IIS 8.0.30"

as the IIS version.

8.0.30 is the **ASP.NET Core/.NET runtime version**.

IIS itself is a Windows component and has its own version.

18. Verify ASP.NET Core Hosting Installation

Run:

```
dotnet --list-runtimes
```

Look for:

```
Microsoft.AspNetCore.App 8.0.x  
Microsoft.NETCore.App 8.0.x
```

Also verify that this file exists:

```
C:\Program Files\IIS\Asp.Net Core Module\V2
```

The ASP.NET Core Module allows IIS to communicate with the ASP.NET Core application.

19. Install IIS

Open:

```
Control Panel  
↓  
Programs  
↓  
Turn Windows features on or off
```

Enable:

```
Internet Information Services
```

At minimum, ensure IIS Management Console and the required Web Server components are installed.

You can also verify IIS using:

```
inetmgr
```

This opens:

```
Internet Information Services (IIS) Manager
```

20. Verify IIS Is Running

Open a browser and enter:

```
http://localhost
```

You should get the IIS welcome page if the default IIS website is running.

21. Create an IIS Application Pool

Open:

IIS Manager

On the left:

Application Pools

Right-click:

Application Pools
↓
Add Application Pool

Enter:

Name:
Dapper_CRUD

Set:

.NET CLR Version:
No Managed Code

Set:

Managed pipeline mode:
Integrated

Then click:

OK

22. Why "No Managed Code"?

ASP.NET Core does not use the old ASP.NET Framework CLR hosting model.

IIS acts as a reverse proxy/front-end.

The ASP.NET Core application runs through the ASP.NET Core Module.

Therefore:

.NET CLR Version = No Managed Code

is the standard configuration for ASP.NET Core IIS hosting.

23. Configure Application Pool Identity

Select:

```
Dapper_CRUD
```

Right-click:

```
Advanced Settings
```

Find:

```
Identity
```

Set it to:

```
ApplicationPoolIdentity
```

This creates the Windows identity:

```
IIS APPPOOL\Dapper_CRUD
```

This identity is important because our application uses Windows Authentication to connect to SQL Server.

24. Configure Application Pool Settings

For a basic deployment:

```
.NET CLR Version = No Managed Code  
Managed Pipeline = Integrated  
Identity = ApplicationPoolIdentity
```

The Application Pool can remain:

```
Start Mode = OnDemand
```

unless the application specifically requires preloading.

25. Give IIS Permission to the Published Folder

The application is located at:

```
D:\IIS-Publish
```

The Application Pool identity needs permission to access this folder.

Right-click:

```
D:\IIS-Publish
```

Select:

```
Properties
  ↓
Security
  ↓
Edit
  ↓
Add
```

Enter:

```
IIS APPPOOL\Dapper_CRUD
```

Click:

```
Check Names
```

It should resolve the account.

Give the account the required permissions, typically:

```
Read & Execute
List folder contents
Read
```

Do not give unnecessary Full Control.

26. Create the IIS Website/Application

In IIS Manager:

```
Sites
```

Right-click:

```
Sites
  ↓
Add Website
```

For example:

```
Site name:
Dapper_CRUD
```

Physical path:

```
D:\IIS-Publish
```

Application Pool:

```
Dapper_CRUD
```

Binding can initially be:

```
Type: http
IP Address: All Unassigned
Port: 80
Host name: blank
```

However, if port 80 is already occupied, use another port, such as:

```
8080
```

Then the application can be accessed as:

```
http://localhost:8080
```

27. Important: Site vs Application

There are two common IIS structures.

Option A — Website

```
Sites
└─ Dapper_CRUD
   └─ Physical Path: D:\IIS-Publish
```

Option B — Application under a Website

For example:

```
Sites
└─ Default Web Site
   └─ api
```

with:

```
Physical Path: D:\IIS-Publish
Application Pool: Dapper_CRUD
```

In your screenshot, you were creating:

```
Site: Dapper_CRUD
Alias: api
```

That is creating an **Application** under the Dapper_CRUD site.

The URL would therefore become something like:

```
http://localhost/api
```

and your API endpoint:

```
http://localhost/api/Employee
```

Make sure the URL you test matches the IIS structure you actually created.

28. Verify Application Pool

In IIS:

| Application Pools

you should see:

| Dapper_CRUD

The status should be:

| Started

If it is stopped:

| Right-click → Start

29. Configure SQL Server Login for IIS

This was the major issue we encountered during your deployment.

Because your application uses:

| Trusted_Connection=True

SQL Server needs to recognize the IIS Application Pool identity.

Open SSMS and go to:

| Security
| ↓
| Logins

Create a login:

| IIS APPPOOL\Dapper_CRUD

The login should exist at the SQL Server level.

30. Create/Verify Database User

Open:

| Databases


```
↓
Dapper
↓
Security
↓
Users
```

There should be a database user corresponding to:

```
IIS APPPOOL\Dapper_CRUD
```

In our case, we verified:

DatabaseUser	ServerLogin

IIS APPPOOL\Dapper_CRUD	IIS APPPOOL\Dapper_CRUD

This is correct.

31. Important: Do Not Use the Wrong User Name

We encountered this error:

```
Cannot find the user 'Dapper', because it does not exist
or you do not have permission.
```

```
The login already has an account under a different user name.
```

This happened because the database user and login names were being confused.

The correct identity is:

```
IIS APPPOOL\Dapper_CRUD
```

not:

```
Dapper
```

The database is:

```
Dapper
```

The IIS identity is:

```
IIS APPPOOL\Dapper_CRUD
```

These are two completely different things.

32. Grant EXECUTE Permission on Stored Procedures

This is the **critical step that caused your HTTP 500 error**.

Your Event Viewer/IIS log showed:

```
The EXECUTE permission was denied on the object 'sp_GetEmployee',  
database 'Dapper', schema 'dbo'.
```

and:

```
The EXECUTE permission was denied on the object 'sp_GetEmployeeById',  
database 'Dapper', schema 'dbo'.
```

This means:

```
IIS → Application → SQL Server
```

was actually working.

SQL Server recognized the IIS account.

But SQL Server was saying:

```
> "This account is allowed to connect to the database, but it is not allowed to execute this stored  
procedure."
```

33. Grant Permission to All Required Procedures

In SSMS, select the `Dapper` database and execute:

```
USE Dapper;  
GO  
  
GRANT EXECUTE ON dbo.sp_AddEmp TO [IIS APPPOOL\Dapper_CRUD];  
GRANT EXECUTE ON dbo.sp_AddManager TO [IIS APPPOOL\Dapper_CRUD];  
GRANT EXECUTE ON dbo.sp_DeleteEmployee TO [IIS APPPOOL\Dapper_CRUD];  
GRANT EXECUTE ON dbo.sp_GetEmployee TO [IIS APPPOOL\Dapper_CRUD];  
GRANT EXECUTE ON dbo.sp_GetEmployeeById TO [IIS APPPOOL\Dapper_CRUD];  
GRANT EXECUTE ON dbo.sp_GetManagerById TO [IIS APPPOOL\Dapper_CRUD];  
GRANT EXECUTE ON dbo.sp_GetManagers TO [IIS APPPOOL\Dapper_CRUD];  
GRANT EXECUTE ON dbo.sp_UpdateEmployee TO [IIS APPPOOL\Dapper_CRUD];  
GRANT EXECUTE ON dbo.sp_UpdatEmployees TO [IIS APPPOOL\Dapper_CRUD];  
GO
```

This is based on the stored procedure list you provided.

34. Better Long-Term Approach: Database Role

For a company application, instead of individually granting permissions to every stored procedure, you can create a database role.

For example:

```
USE Dapper;
GO

CREATE ROLE DapperApiRole;
GO

GRANT EXECUTE TO DapperApiRole;
GO

ALTER ROLE DapperApiRole
ADD MEMBER [IIS APPPOOL\Dapper_CRUD];
GO
```

However, whether GRANT EXECUTE on the whole database is appropriate depends on your company's security policy.

More restrictive approach

Grant only the procedures required by the application:

```
GRANT EXECUTE ON dbo.sp_GetEmployee TO DapperApiRole;
GRANT EXECUTE ON dbo.sp_GetEmployeeById TO DapperApiRole;
GRANT EXECUTE ON dbo.sp_AddEmp TO DapperApiRole;
GRANT EXECUTE ON dbo.sp_UpdateEmployee TO DapperApiRole;
GRANT EXECUTE ON dbo.sp_DeleteEmployee TO DapperApiRole;
```

For production systems, **least privilege is preferable**.

35. Verify SQL Permission

You can check the permissions using:

```
USE Dapper;
GO

SELECT
    dp.state_desc,
    dp.permission_name,
    OBJECT_NAME(dp.major_id) AS ObjectName
FROM sys.database_permissions dp
WHERE dp.grantee_principal_id =
    DATABASE_PRINCIPAL_ID('IIS APPPOOL\Dapper_CRUD');
```

You should see permissions such as:

```
GRANT  
EXECUTE  
sp_GetEmployee
```

etc.

36. Restart the IIS Application

After changing configuration or permissions, restart the Application Pool.

Go to:

```
IIS Manager  
↓  
Application Pools  
↓  
Dapper_CRUD
```

Right-click:

```
Recycle
```

If **Recycle** isn't visible in your particular IIS view, you can:

```
Stop
```

and then:

```
Start
```

You can also restart IIS from an elevated Command Prompt:

```
iisreset
```

Important

You normally do **not** need to restart IIS after every SQL permission change. But recycling the application pool is a clean way to ensure the application is restarted.

37. Test the API

Your controller has:

```
[Route("api/[controller]")]
```

and the controller is:

```
EmployeeController
```

Therefore the base route is:

```
| /api/Employee
```

38. API Endpoints

Your controller contains the following endpoints.

GET all employees

```
| GET /api/Employee
```

Example:

```
| http://localhost/api/Employee
```

If using port 8080:

```
| http://localhost:8080/api/Employee
```

GET employee by ID

```
| GET /api/Employee/{id}
```

Example:

```
| http://localhost/api/Employee/1
```

POST employee

```
| POST /api/Employee
```

Request body depends on your `Employee` model.

Example:

```
| {  
  "employeeName": "John",  
  "salary": 50000  
}
```

Use the actual properties of your model.

PUT employee

```
| PUT /api/Employee/{id}
```

Example:

```
PUT /api/Employee/1
```

DELETE employee

```
DELETE /api/Employee/{id}
```

Example:

```
DELETE /api/Employee/1
```

39. Test Using Browser

For GET:

```
http://localhost/api/Employee
```

If everything is working, you should receive JSON.

For example:

```
[
  {
    "employeeId": 1,
    "employeeName": "John"
  },
  {
    "employeeId": 2,
    "employeeName": "Smith"
  }
]
```

40. Test Using Swagger

If Swagger is enabled in your application, access:

```
http://localhost/swagger
```

or:

```
http://localhost:8080/swagger
```

depending on the IIS binding.

Swagger allows you to test:

```
GET
POST
PUT
DELETE
```

without manually constructing requests.

41. Test Using Postman

For example:

GET

```
GET http://localhost/api/Employee
```

GET by ID

```
GET http://localhost/api/Employee/1
```

POST

```
POST http://localhost/api/Employee
```

Set:

```
Body
  ↓
raw
  ↓
JSON
```

Then provide the employee JSON.

42. Common HTTP 500 Error We Encountered

Your original error was:

```
HTTP ERROR 500
```

The browser only showed:

```
localhost is currently unable to handle this request.
```

That message alone is not enough to identify the problem.

We checked the application logs and Event Viewer.

The actual error was:

```
Microsoft.Data.SqlClient.SqlException
```

with:

```
Error Number: 229
```

and:

```
The EXECUTE permission was denied on the object  
'sp_GetEmployee'
```

This is a SQL Server permission issue.

43. How to Diagnose IIS HTTP 500

When an IIS API gives:

```
HTTP 500
```

do not immediately change IIS settings randomly.

Follow this sequence:

```
Browser  
↓  
IIS  
↓  
ASP.NET Core  
↓  
Service  
↓  
Dapper  
↓  
SQL Server
```

Find the exact layer where the error occurs.

44. Check Application Logs

Your application was writing logs showing:

```
Hosting environment: Production  
Content root path: D:\IIS-Publish
```

Then:

```
Microsoft.Data.SqlClient.SqlException
```

This tells us:

IIS is working

because the application started.

ASP.NET Core is working

because the request reached the controller.

Dapper is working

because Dapper attempted to execute SQL.

SQL connection is working

because SQL Server returned a permission error.

SQL permissions were the problem

because:

```
EXECUTE permission was denied
```

This is an important diagnostic principle for future deployments.

45. Check Event Viewer

Open:

```
Event Viewer
```

Go to:

```
Windows Logs  
↓  
Application
```

Look for errors related to:

```
Microsoft.AspNetCore.Server.IIS.Core.IISHttpServer
```

In your case, Event Viewer confirmed:

```
RequestPath: /api/Employee
```

and:

```
The EXECUTE permission was denied on the object  
'sp_GetEmployee'
```

Therefore Event Viewer confirmed the same SQL permission problem.

46. Check IIS Site Status

In IIS Manager:

| Sites

Make sure:

| Dapper_CRUD

is:

| Started

If stopped:

| Right-click → Start

47. Check Application Pool Status

Go to:

| Application Pools

Verify:

| Dapper_CRUD

is:

| Started

If it repeatedly stops, check:

| Event Viewer

because repeated Application Pool crashes usually indicate a configuration/runtime/application problem.

48. Check Physical Path

IIS should point to the **published folder**:

| D:\IIS-Publish

not the source project folder.

Verify:

| IIS

- Sites
- Dapper_CRUD
- Basic Settings

Physical Path:

D:\IIS-Publish

49. Check web.config

The published folder should contain:

web.config

This is normally generated automatically by `dotnet publish`.

It allows IIS to use the ASP.NET Core Module to launch the application.

Do not manually create a random `web.config` unless you understand the required configuration.

50. Check Connection String

Verify:

```
"ConnectionStrings": {  
  "DbConn":  
    "Server=localhost\\SQLEXPRESS;Database=Dapper;Trusted_Connection=True;TrustServerCertificate=True;"  
}
```

Make sure:

Server
Database
Authentication

are correct.

One previous error we encountered was:

The `ConnectionString` property has not been initialized.

That is a **different problem** from the current SQL permission error.

It usually indicates that the application did not correctly retrieve the configured connection string.

51. Important Deployment Rule

After changing:

```
appsettings.json
Program.cs
Controllers
Services
Models
```

you need to publish the updated application again.

For example:

```
dotnet publish -c Release -o D:\IIS-Publish
```

Then recycle the Application Pool.

Do not change the source code and assume IIS automatically gets the changes.

52. Recommended Update/Deployment Process

For future releases:

```
1. Get latest source code
    ↓
2. Build
    ↓
3. Test
    ↓
4. Publish
    ↓
5. Backup existing IIS application
    ↓
6. Copy new published files
    ↓
7. Verify configuration
    ↓
8. Recycle Application Pool
    ↓
9. Test API
```

If using Git:

```
git pull
```

Then:

```
dotnet restore
```

Then:

```
dotnet build
```

Then:

```
dotnet publish -c Release -o D:\IIS-Publish
```

53. Recommended Production Backup

Before replacing an existing deployment:

```
D:\IIS-Publish
```

take a backup.

For example:

```
D:\IIS-Backup\2026-08-23\
```

Then copy the existing application:

```
D:\IIS-Publish
```

↓

```
D:\IIS-Backup\2026-08-23
```

This allows rollback if the new deployment fails.

54. Recommended Production Folder Structure

A cleaner company setup could be:

```
D:\
├── Applications
│   ├── Dapper_CRUD
│   │   └── Current
├── Backups
│   ├── Dapper_CRUD
│   │   ├── 2026-08-20
│   │   └── 2026-08-23
└── Logs
    └── Dapper_CRUD
```

Your current setup:

```
D:\IIS-Publish
```

is perfectly fine for learning/small internal deployment.

For a larger production environment, versioned deployment folders are better because rollback becomes straightforward.

55. Complete Command Summary

For documentation, this is the condensed command sequence.

Check Git

```
git --version
```

Clone repository

```
cd D:\Projects  
git clone <REPOSITORY-URL>
```

Enter project

```
cd D:\Projects\Dapper-API-React\Dapper_CRUD
```

Check .NET

```
dotnet --version  
dotnet --list-sdks  
dotnet --list-runtimes
```

Restore

```
dotnet restore
```

Build

```
dotnet build
```

Create deployment folder

```
mkdir D:\IIS-Publish
```

Publish

```
dotnet publish -c Release -o D:\IIS-Publish
```

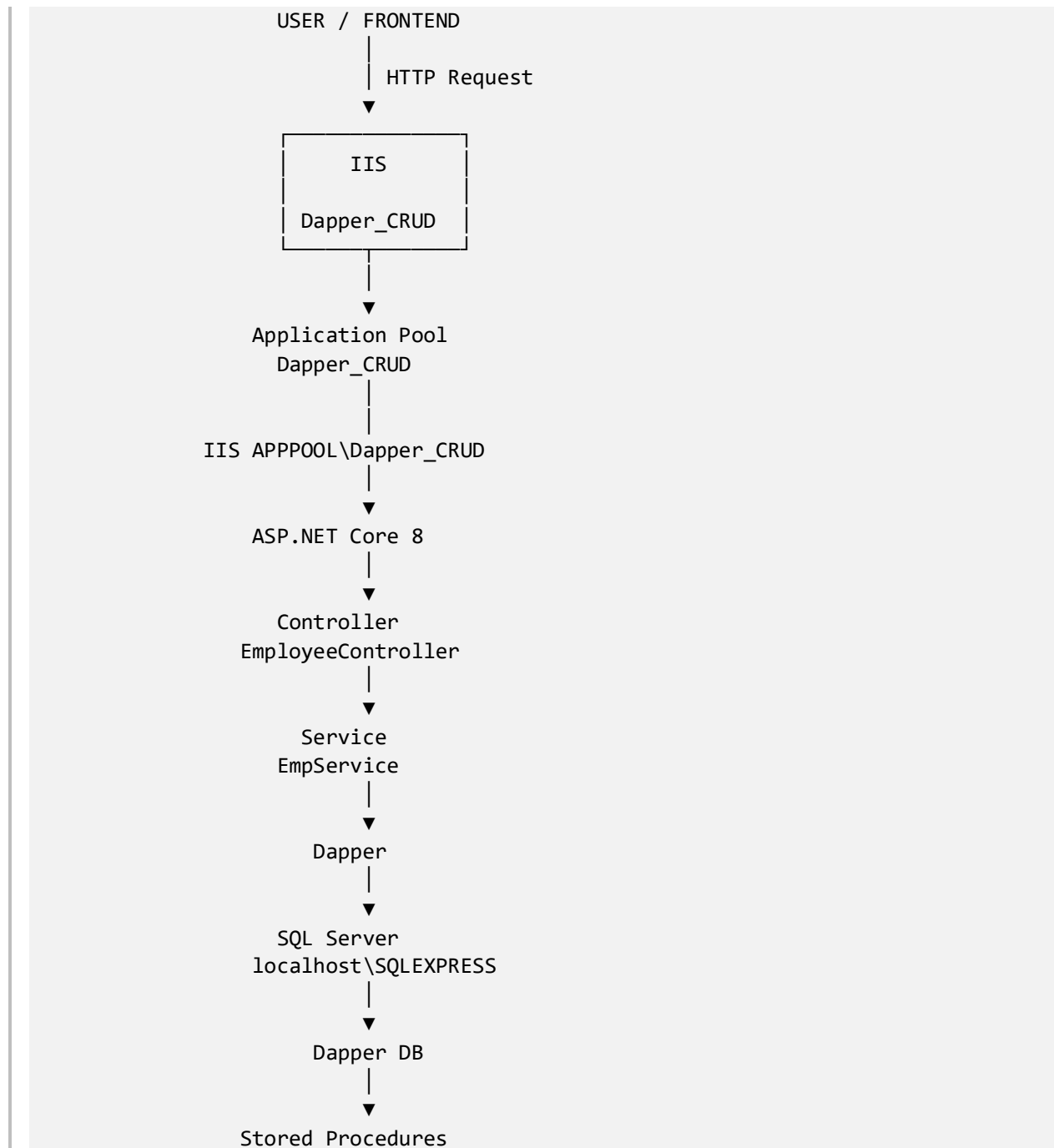
Restart IIS

Run Command Prompt/PowerShell **as Administrator**:

```
iisreset
```

56. Complete Deployment Architecture

The final architecture looks like this:



57. Final Deployment Checklist

Before declaring the deployment successful, verify each item:

Application

- * ☐ Source code downloaded/cloned
- * ☐ Correct .NET version identified
- * ☐ dotnet restore successful
- * ☐ dotnet build successful
- * ☐ dotnet publish successful
- * ☐ Published files exist in D:\IIS-Publish

.NET/IIS

- * ☐ .NET runtime installed
- * ☐ ASP.NET Core Hosting Bundle installed
- * ☐ IIS installed
- * ☐ IIS site created
- * ☐ Application Pool created
- * ☐ Application Pool uses No Managed Code
- * ☐ Application Pool uses ApplicationPoolIdentity
- * ☐ Application Pool is started
- * ☐ IIS physical path points to D:\IIS-Publish
- * ☐ web.config exists

File Permissions

- * ☐ IIS APPPOOL\Dapper_CRUD can read/execute the published folder

SQL Server

- * ☐ SQL Server is running
- * ☐ Dapper database exists
- * ☐ Required tables exist
- * ☐ Required stored procedures exist
- * ☐ SQL Server login exists:

IIS APPPOOL\Dapper_CRUD

- * ☐ Database user exists:

IIS APPPOOL\Dapper_CRUD

* [] Required `EXECUTE` permissions are granted

Testing

* [] GET all employees works

* [] GET employee by ID works

* [] POST works

* [] PUT works

* [] DELETE works

* [] Swagger works, if enabled

* [] No HTTP 500

* [] Event Viewer has no relevant application errors

The key lesson from our deployment issue

The most important part of the troubleshooting we did is this:

```
HTTP 500
↓
Check IIS/Application logs
↓
SQL Exception 229
↓
EXECUTE permission denied
↓
Check IIS identity
↓
IIS APPPOOL\Dapper_CRUD
↓
Create SQL Login
↓
Create/verify Dapper database user
↓
Grant EXECUTE permission
↓
Recycle Application Pool
↓
Test API
```

So **IIS itself was not fundamentally broken** in your case. The application was successfully starting and reaching SQL Server. The failure was that the IIS Application Pool identity did not have permission to execute the stored procedures.

For a company SOP, I would keep this document as the **standard deployment procedure**, and add a separate "**IIS Deployment Troubleshooting Guide**" containing the common 500, 403, 404, 405, connection-string, runtime, permissions, and SQL authentication errors.